北京电子科技学院（BESTI）

实　　验　　报　　告

课程：密码系统设计　　班级：2414

姓名：穆弈涵　　学号：20241422

成绩：　　指导教师：娄嘉鹏　　实验日期：10 月 8 日

实验密级：无　　预习程度：部分预习

实验时间：3,4 节　仪器组次：无　　必修/选修：必修

实验序号：实验一

实验名称：嵌入式开发基础

实验目的与要求：掌握 Linux 系统使用与开发方法，掌握 OpenSSL/GmSSL 命令用法，掌握常见商用密码算法 SM3/SM4/SM2 的使用，并实现带签名的数字信封协议。

实验仪器：

名称	型号	数量
计算机		1

实验内容、步骤与体会（附纸）

1. 实验内容

本次实验是《密码系统设计》实验一"嵌入式开发基础"的 **1-3 学时实践部分（30分）**，在 openEuler 24.03 LTS 虚拟机中完成，具体内容包括：

1. 搭建 Linux（openEuler）实验环境，安装并核对 OpenSSL、GmSSL 开发工具；
2. 使用 **OpenSSL 命令** 实践国产密码算法：随机数、SM3 摘要、SM4 加解密、SM2 签名验签与加解密；
3. 使用 **GmSSL 命令** 实践上述国产密码算法，并与 OpenSSL 结果对照；
4. 两人一组，使用 **OpenSSL 命令** 实现带签名的数字信封协议（Alice 发送 → Bob 接收）；
5. 两人一组，使用 **GmSSL 命令** 实现带签名的数字信封协议（Bob 发送 → Alice 接收）；
6. 将代码与文档托管到 GitHub，并提交本次实验相关的 `git log` 运行结果。

实验中身份替换：**Alice = 20241422 穆弈涵（本人）**，**Bob = 20241412 魏继博（搭档）**。 每完成一项实践均执行一次 `git commit`。

2. 实验步骤

2.1 步骤 1：搭建 openEuler 实验环境

2.1.1 操作内容

1. 在 VMware 中新建 **openEuler 24.03 LTS-SP4** 虚拟机（图形界面，主机名 `localhost`，guest 用户 `20241422myh`）。
2. 解决宿主机与虚拟机之间的剪贴板/共享目录问题（openEuler 默认未装 VMware 工具）： 编写并执行一次性脚本 `scripts/setup-vm.sh`，安装 `open-vm-tools-desktop`、开启 `/etc/fuse.conf` 的 `user_allow_other`，并以 `uid/gid` 挂载共享文件夹 `lab1` ↔ 本仓库目录。
3. 设置 **学号水印**：把 `PS1='[20241422 穆弈涵 \u@\h \W]\$ '` 追加进 `~/.bashrc`（写进 `.bashrc` 而非脚本里的 `export`，否则重开终端失效）。这样每条命令回显前都带本人学号，满足“截图需能看出是本人所做”的要求。
4. 记录系统信息：`cat /etc/os-release | head -2`、`uname -r`、`echo $XDG_CURRENT_DESKTOP`。
5. 核对 OpenSSL：`openssl version -a`；列举 SM3/SM4 算法：`openssl list -digest-algorithms | grep -i sm3`、`openssl list -cipher-algorithms | grep -i sm4`。
6. 源码编译安装 **GmSSL 3.2.0**：从 GitHub 官方仓库下载 `v3.2.0` 源码包，经共享目录送入虚拟机，依次执行 `cmake .. -DCMAKE_BUILD_TYPE=Release` → `make -j$(nproc)` → `sudo make install` →

`sudo ldconfig`。安装后用 `gmssl version`、`gmssl rand -hex -outlen 16`、`echo -n abcd | gmssl sm3` 验证。

2.1.2 操作结果

系统为 openEuler 24.03 (LTS-SP4)，内核 6.6.0，桌面环境 UKUI；OpenSSL 为 3.5.6，GmSSL 为 3.2.0。共享目录 `lab1` 挂载成功（`ls /mnt/hgfs/lab1` 可见 `docs README.md screenshots scripts`），双向粘贴可用；终端提示符带学号水印。环境各项检查全部通过，见下图。

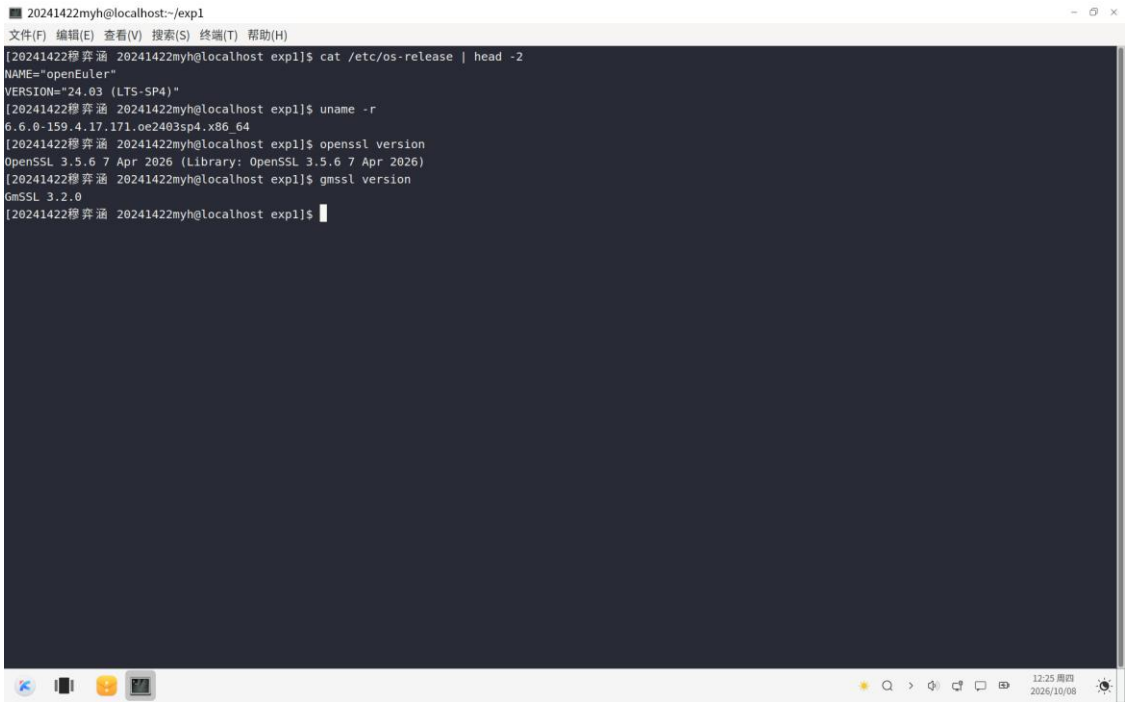


图 01 环境总览：系统信息、OpenSSL/GmSSL 版本、共享目录与学号水印提示符

2.2 步骤 2：OpenSSL 命令实践（随机数 / SM3 / SM4 / SM2）

2.2.1 操作内容

工作目录为 `~/exp1`。按题面第 1 条“实践 OpenSSL 相关内容”，逐项实践：

1. **版本总览**：`openssl version -a | head -4`。
2. **随机数**：`openssl rand -hex 16`。
3. **SM3 摘要（双引擎对照）**：`echo -n '20241422 穆弈涵' > t.txt` → `openssl dgst -sm3 t.txt`，再用 GmSSL `gmssl sm3 -in t.txt` 对照同一输入的摘要。
4. **SM4-CBC 加解密往返**：`K=$(openssl rand -hex 16); IV=$(openssl rand -hex 16)` 生成密钥与 IV；`openssl enc -sm4-cbc -K $K -iv $IV -in plain1.txt -out sm4c.bin` 加密，`openssl enc -sm4-cbc -d ...` 解密，`cmp` 校验一致，`od -A d -t x1 sm4c.bin` 观察密文。
5. **SM2 密钥生成与签名验签**：`openssl genpkey -algorithm SM2 -out sm2a.key` → `openssl pkey -in sm2a.key -pubout -out sm2a.pub`；签名 `openssl pkeyutl -sign -in t.txt -inkey sm2a.key -rawin -digest sm3 -out t.sig`；验签 `openssl pkeyutl -verify -in t.txt -pubin -inkey sm2a.pub -rawin -digest sm3 -sigfile t.sig`；并构造**篡改负例**（修改原文后再验签）。
6. **SM2 加解密**：`openssl pkeyutl -encrypt -in k16.bin -inkey sm2b.pub -pubin -out k16.enc`，`openssl pkeyutl -decrypt -in k16.enc -inkey sm2b.key -out k16.dec`，`cmp` 校验。

2.2.2 操作结果

- 随机数 `openssl rand -hex 16` 输出 32 个 hex 字符（16 字节）；SM3 摘要 `dfc1bb19…76ad028ff` 与 GmSSL 输出**逐字符一致**。

- SM4-CBC：明文 30 字节 → 密文 32 字节，**PKCS#7 补齐 2 字节**；解密后 `cmp` 一致（解密一致）。
- SM2 签名验签 `Signature Verified Successfully`；篡改原文后验签 **`Signature Verification Failure, exit=1`**。
- SM2 加解密往返一致（SM2 加解密一致）。

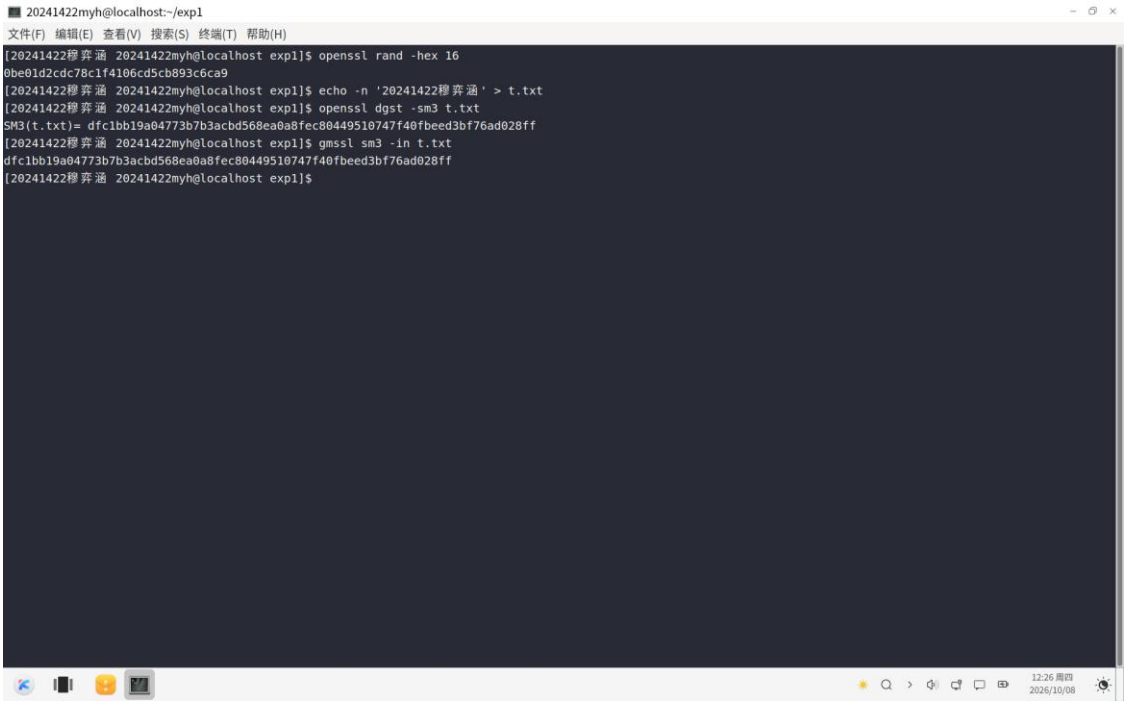


图 02 随机数与 SM3 双引擎对照

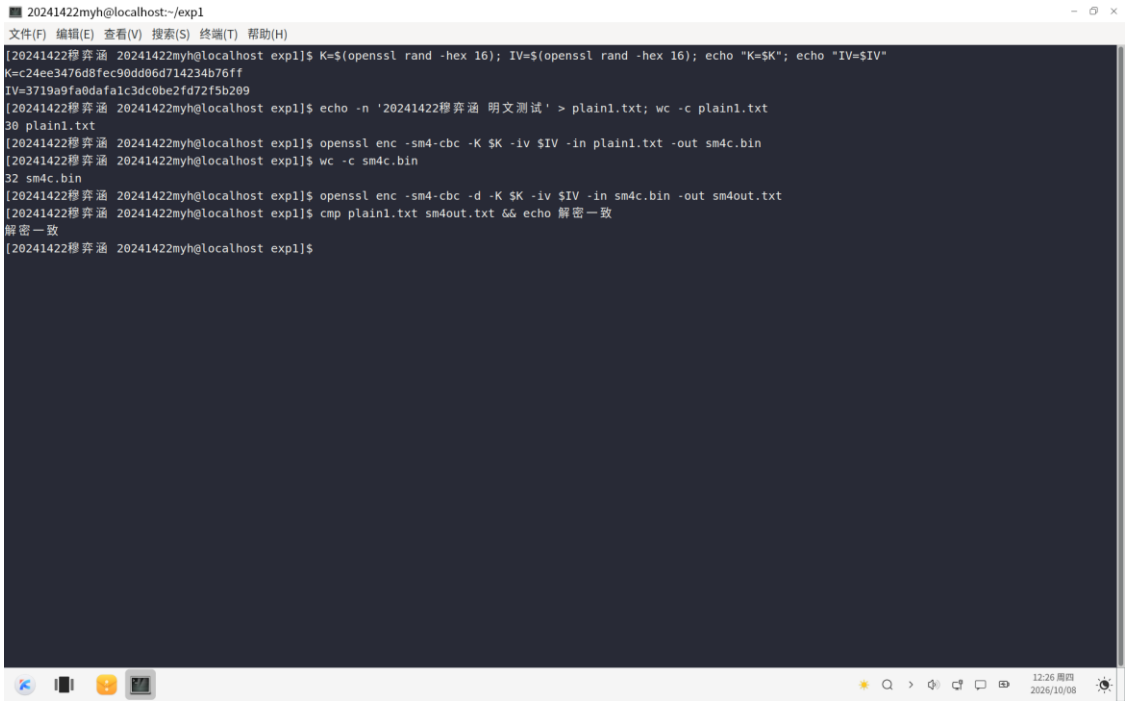


图 03 SM4-CBC 加解密往返（30→32 字节，PKCS#7 补齐）

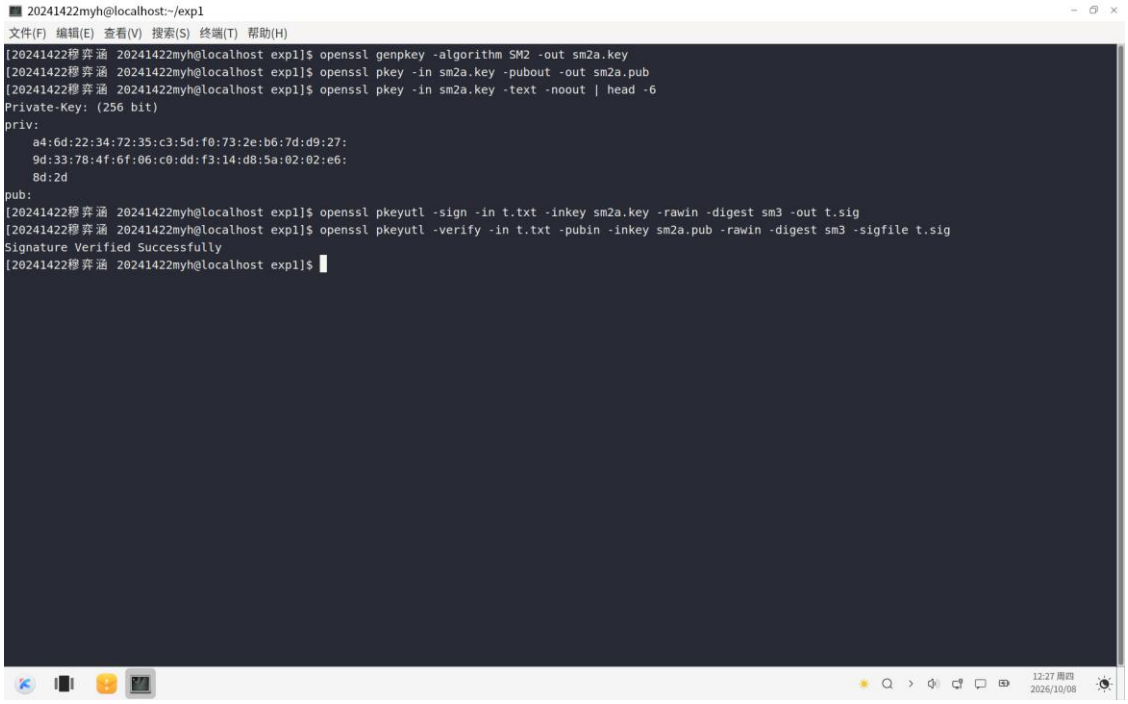


图 04 SM2 签名与验签

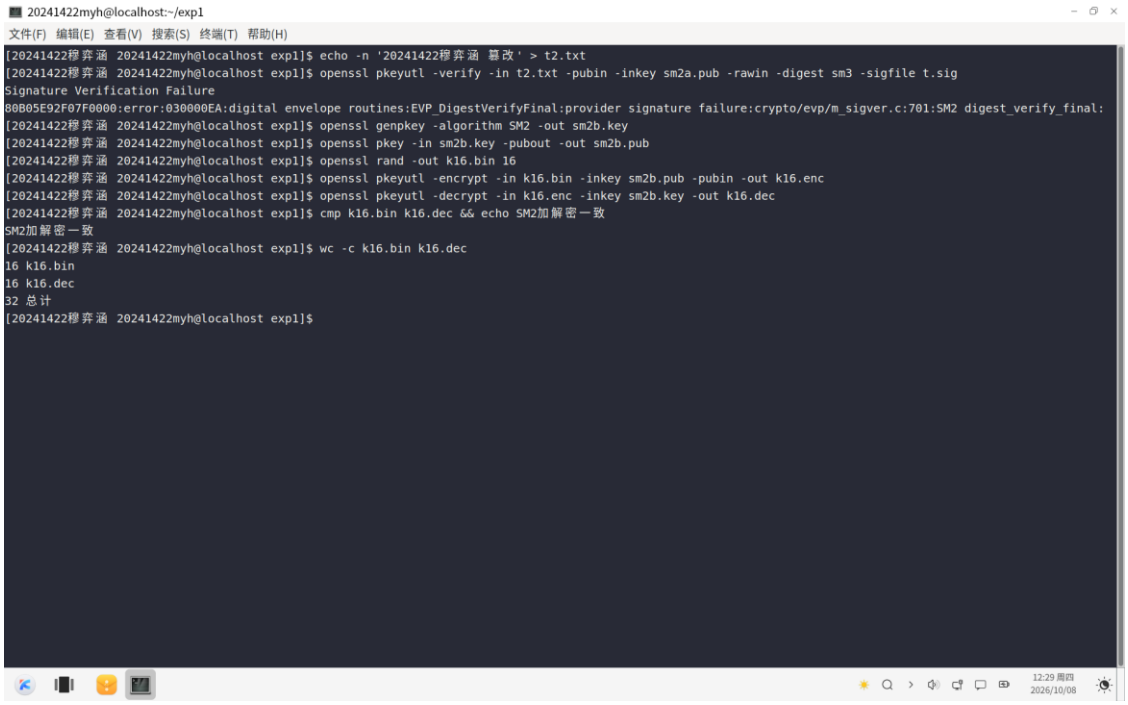


图 05 SM2 篡改负例与 SM2 加解密

2.3 步骤 3：GmSSL 命令实践（SM4 / SM2）

2.3.1 操作内容

依据题面第 2 条“实践 GmSSL 相关内容”。**重要前置**：GmSSL 3.2.0 的命令行与旧文档/题面差异很大，所有命令先读源码 `tools/*.c` 的 usage 核对再执行。逐项实践：

1. 版本：`gmssl version`。
2. **SM4-CBC**：`K2=$(gmssl rand -hex -outlen 16); IV2=$(gmssl rand -hex -outlen 16)`，`gmssl sm4_cbc -encrypt -key $K2 -iv $IV2 -in plain2.txt -out sm4c2.bin`，`gmssl sm4_cbc -decrypt ...`，`cmp` 校验。

3. **SM2 密钥与签名验签**： `gmssl sm2keygen -pass 123456 -out bob.key -pubout bob.pub`； `gmssl sm2sign -key bob.key -pass 123456 -id 1234567812345678 -in msg.txt -out msg.sig`；`gmssl sm2verify -pubkey bob.pub -id 1234567812345678 -in msg.txt -sig msg.sig`； 并构造篡改负例。
4. **SM2 加解密**： `gmssl sm2encrypt -pubkey bob.pub -in k16b.bin -out k16b.enc`， `gmssl sm2decrypt -key bob.key -pass 123456 -in k16b.enc -out k16b.dec`，`cmp` 校验。

2.3.2 操作结果

- `gmssl sm4_cbc` 与 OpenSSL 完全同参数：明文 30 字节 → 密文 32 字节，`cmp` 一致。
- SM2 签名验签 `verify : success`；篡改消息后 GmSSL 报 **内部错误 + 源码行号**（`sm2_sign.c:265/671`），**`exit=1`**（失败判定以退出码为准）。
- SM2 加解密往返一致；密文尺寸 **16 字节明文 → 123 字节密文**，与 OpenSSL 相同。

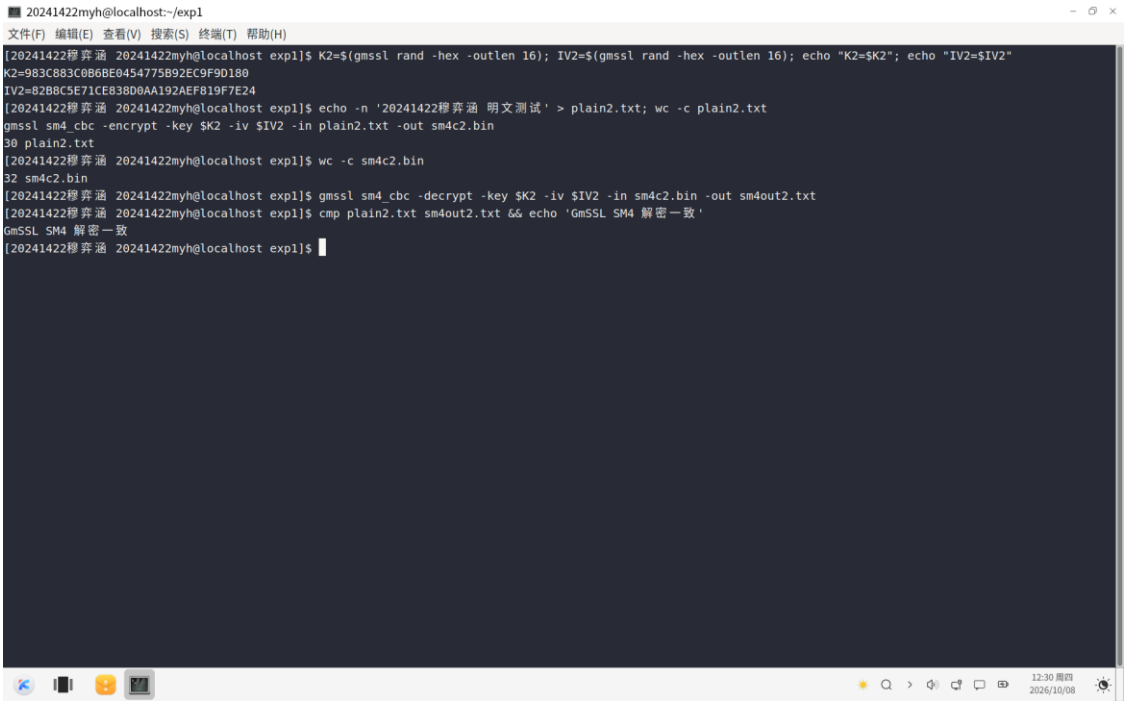


图 06 GmSSL SM4-CBC 往返

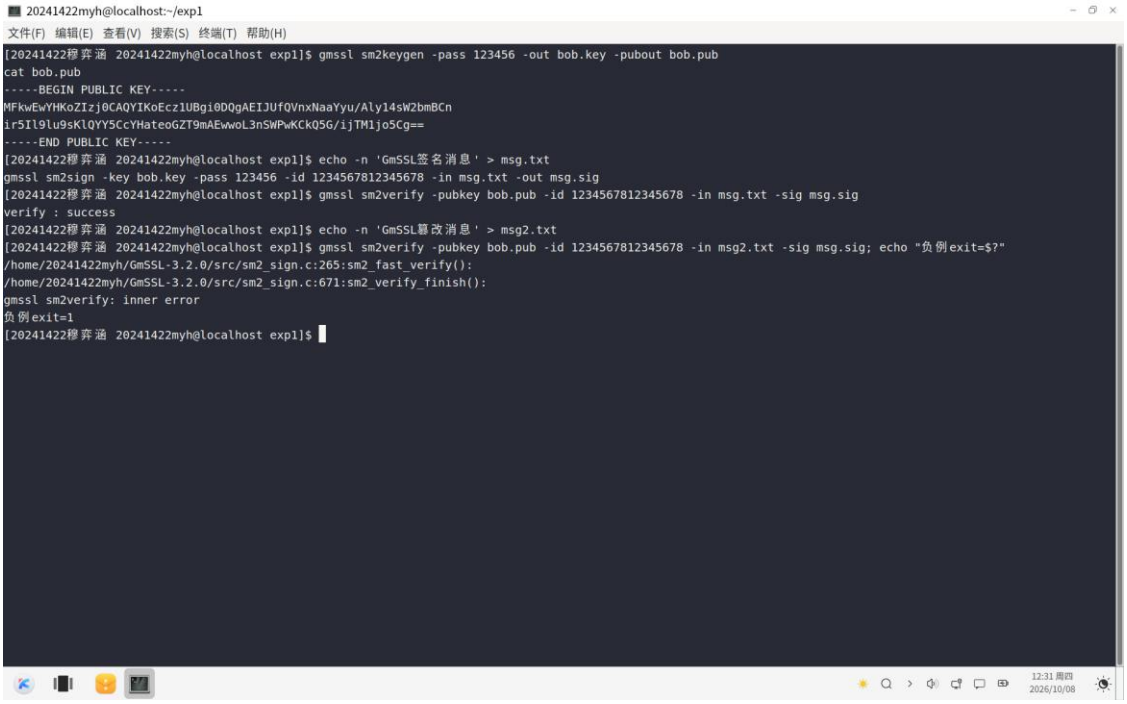


图 07 GmSSL SM2 签名验签与篡改负例

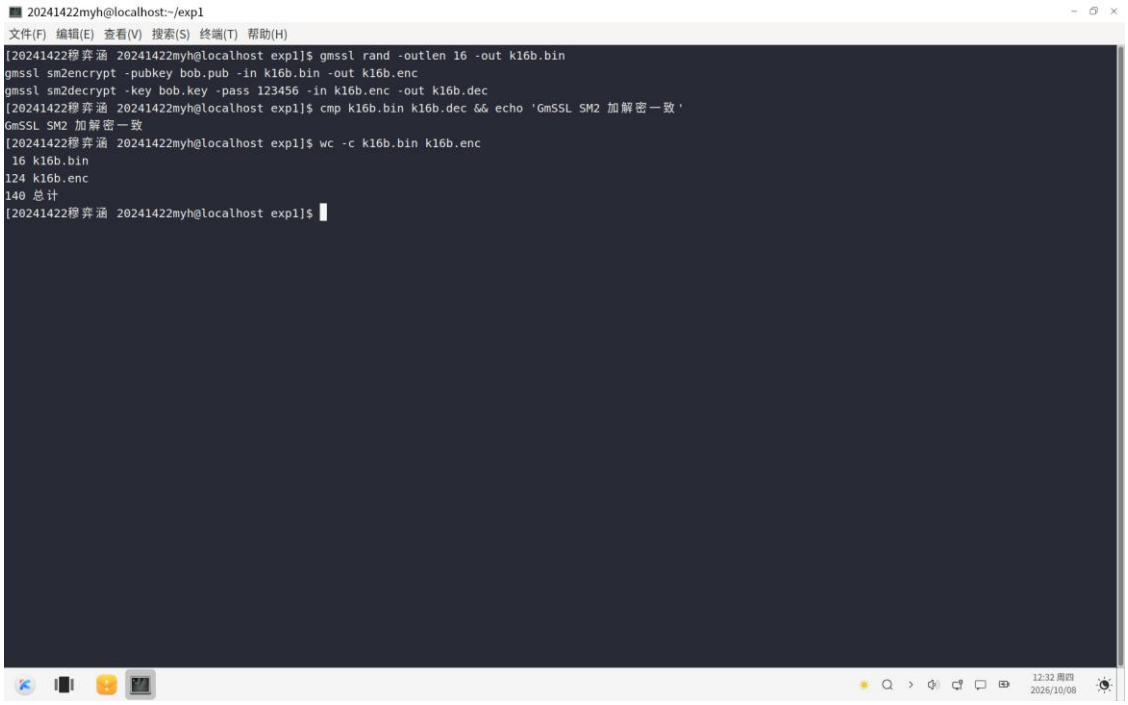


图 08 GmSSL SM2 加解密

2.4 步骤 4：数字信封协议（OpenSSL 命令实现，Alice 发送 → Bob 接收）

2.4.1 操作内容

题面协议：Alice 与 Bob 各生成 SM2 公私钥对并交换公钥；Alice 产生 16 字节 SM4 密钥 k，对明文 P 加密得 C，用 Bob 公钥加密 k 得 KC，用自己私钥对 C 签名得 S1，发送 C||KC||S1；Bob 验签、解密 KC 取 k、解密 C 还原 P。

以 alice/、bob/ 两个目录 + cp 模拟信道传输。Alice 侧：

```
openssl genpkey -algorithm SM2 -out alice/ska.key
openssl pkey -in alice/ska.key -pubout -out alice/pka.pub
openssl genpkey -algorithm SM2 -out bob/skb.key
openssl pkey -in bob/skb.key -pubout -out bob/pkb.pub
echo -n '20241422 穆弈涵' > alice/plain.txt
k=$(openssl rand -hex 16)                # k=622422ca19bc93c88d234d71
```

```
b621b8cf
openssl enc -sm4-cbc -K $k -iv 00000000000000000000000000000000 -in a
lice/plain.txt -out alice/C.txt
openssl pkeyutl -encrypt -in alice/k.txt -inkey bob/pkb.pub -pubin -o
ut alice/KC.bin
openssl pkeyutl -sign -in alice/C.txt -inkey alice/ska.key -rawin -di
gest sm3 -out alice/S1.bin
cp alice/pka.pub bob/ ; cp alice/C.txt alice/KC.bin alice/S1.bin bob/
```

Bob 侧:

```
openssl pkeyutl -verify -in bob/C.txt -pubin -inkey bob/pka.pub -rawi
n -digest sm3 -sigfile bob/S1.bin
openssl pkeyutl -decrypt -in bob/KC.bin -inkey bob/skb.key -out bob/k
_recv.txt
diff alice/k.txt bob/k_recv.txt
openssl enc -sm4-cbc -d -K $krecv -iv 00000000000000000000000000000000
0 -in bob/C.txt -out bob/recv_plain.txt
cmp alice/plain.txt bob/recv_plain.txt
```

协议未定义处显式补全：**SM4-CBC 的 IV 固定为全零**，公钥经"协议外提前交换"。

2.4.2 操作结果

各分量尺寸与来历：明文 17 B（学号 8 数字 + 3 汉字×3 = 17）；C 32 B；KC 139 B；S1 71 B。其中会话密钥 k 为 16 字节，此处以其 32 字符十六进制文本形式封装进 KC，故 KC 为 139 B（直接封装 16 字节原始密钥时为 123 B，见 2.3.2 实测）。 Bob 侧 `Signature Verified Successfully`；`diff k` 一致（会话密钥被正确取回）；`cmp plain` 一致（明文完整送达）——信封闭环。

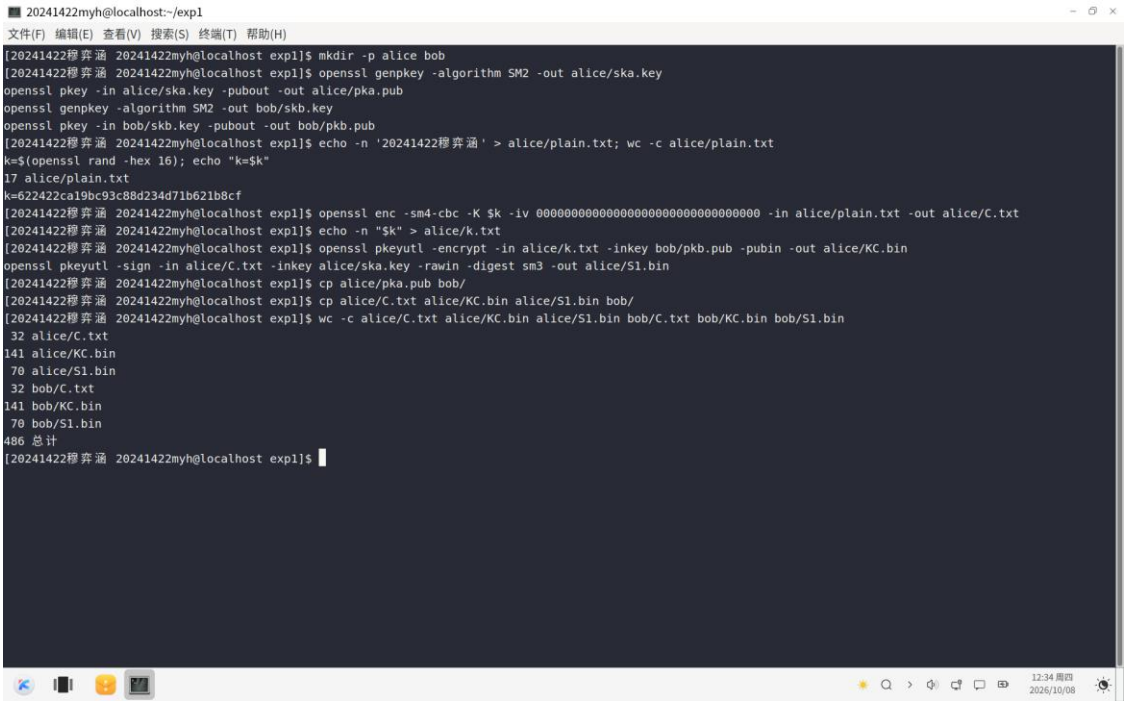


图 09 数字信封 OpenSSL：Alice 侧打包（C/KC/S1）

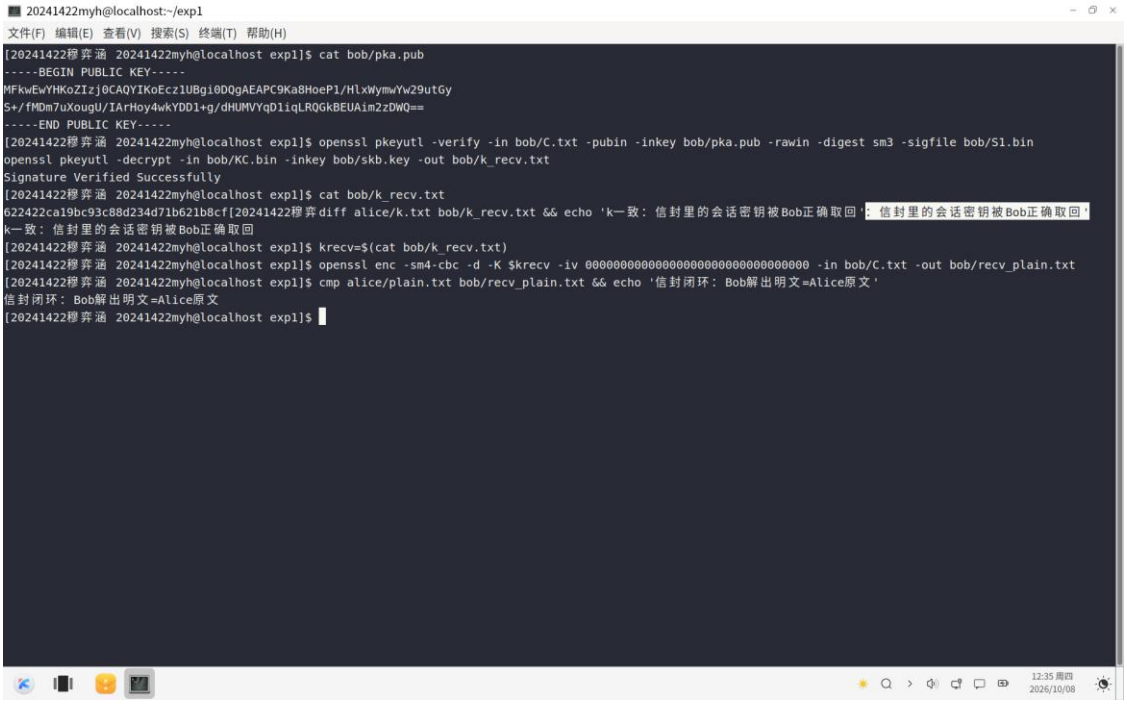


图 10 数字信封 OpenSSL：Bob 侧验签与解信封闭环

2.5 步骤 5：数字信封协议（GmSSL 命令实现，Bob 发送 → Alice 接收）

2.5.1 操作内容

与步骤 4 同一协议、**方向反转**，改用 GmSSL 命令。以 t4/bob/、t4/alice/ 两目录模拟信道。Bob 侧：

```
gmssl sm2keygen -pass 123456 -out t4/bob/skb.key -pubout t4/bob/pkb.pub
gmssl sm2keygen -pass 123456 -out t4/alice/ska.key -pubout t4/alice/pka.pub
printf '%s' '20241412 魏继博' > t4/bob/plain.txt
k=$(gmssl rand -hex -outlen 16)          # k=4F880D69DBFEA53AF841476EA9EE093A
gmssl sm4_cbc -encrypt -key $k -iv 00000000000000000000000000000000 -in t4/bob/plain.txt -out t4/bob/C.txt
gmssl sm2encrypt -pubkey t4/alice/pka.pub -in t4/bob/k.txt -out t4/bob/KC.bin
gmssl sm2sign -key t4/bob/skb.key -pass 123456 -id 1234567812345678 -in t4/bob/C.txt -out t4/bob/S1.bin
cp t4/bob/pkb.pub t4/alice/ ; cp t4/bob/C.txt t4/bob/KC.bin t4/bob/S1.bin t4/alice/
```

Alice 侧：

```
gmssl sm2verify -pubkey t4/alice/pkb.pub -id 1234567812345678 -in t4/alice/C.txt -sig t4/alice/S1.bin
gmssl sm2decrypt -key t4/alice/ska.key -pass 123456 -in t4/alice/KC.bin -out t4/alice/k_recv.txt
diff t4/bob/k.txt t4/alice/k_recv.txt
gmssl sm4_cbc -decrypt -key $krecv -iv 00000000000000000000000000000000 -in t4/alice/C.txt -out t4/alice/recv_plain.txt
cmp t4/bob/plain.txt t4/alice/recv_plain.txt
```

并构造篡改负例：改动 C 的第 5 字节后重新验签。整套命令固化为

scripts/task4-envelope-gmssl.sh，可复现。

2.5.2 操作结果

各分量尺寸：明文 17 B；C 32 B；KC **141 B**；S1 71 B。Alice 侧 verify：success；diff k、cmp plain 均一致（闭环）。篡改 C 后验签输出 **inner error + 源码行号，exit=1**。 其中 KC 的 141 B 与 OpenSSL 的 139 B **不同**——经 od 字节级核对，差异在 DER 结构（GmSSL 为 SEQUENCE{INTEGER,INTEGER,OCTET,OCTET}，OpenSSL 为 C1||C3||C2），算法本身一致。两者封装进 KC 的均为 k 的 32 字符十六进制文本（非 16 字节原始密钥）。

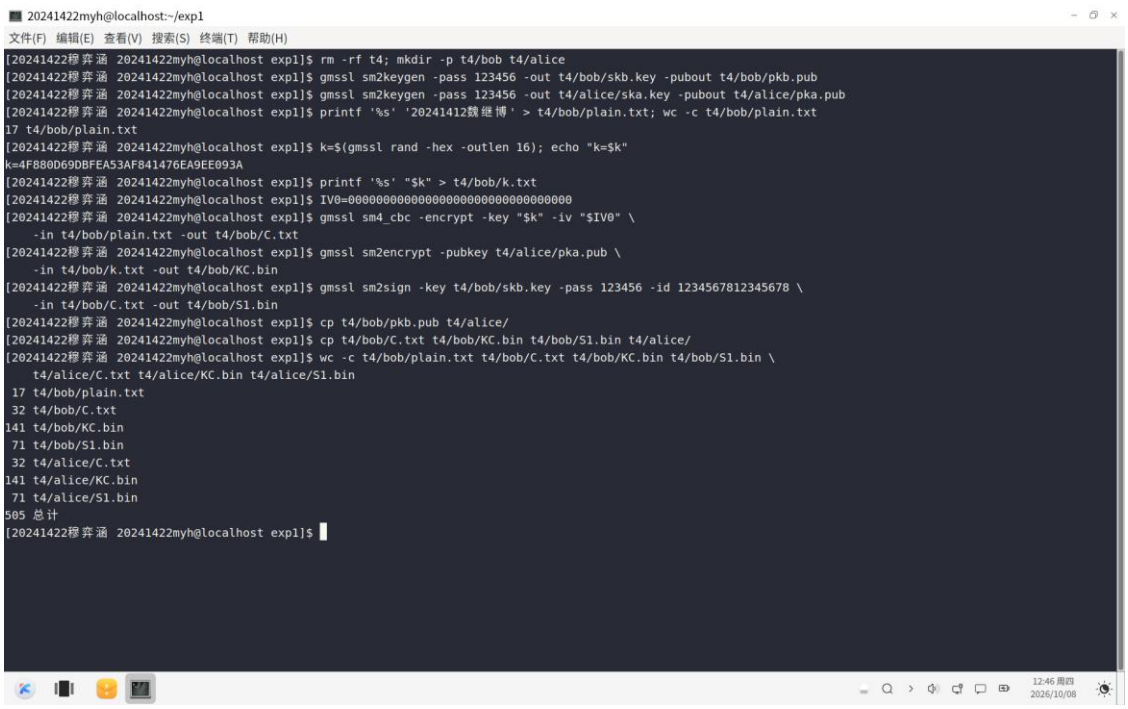


图 11 数字信封 GmSSL：Bob 侧打包（Bob→Alice）

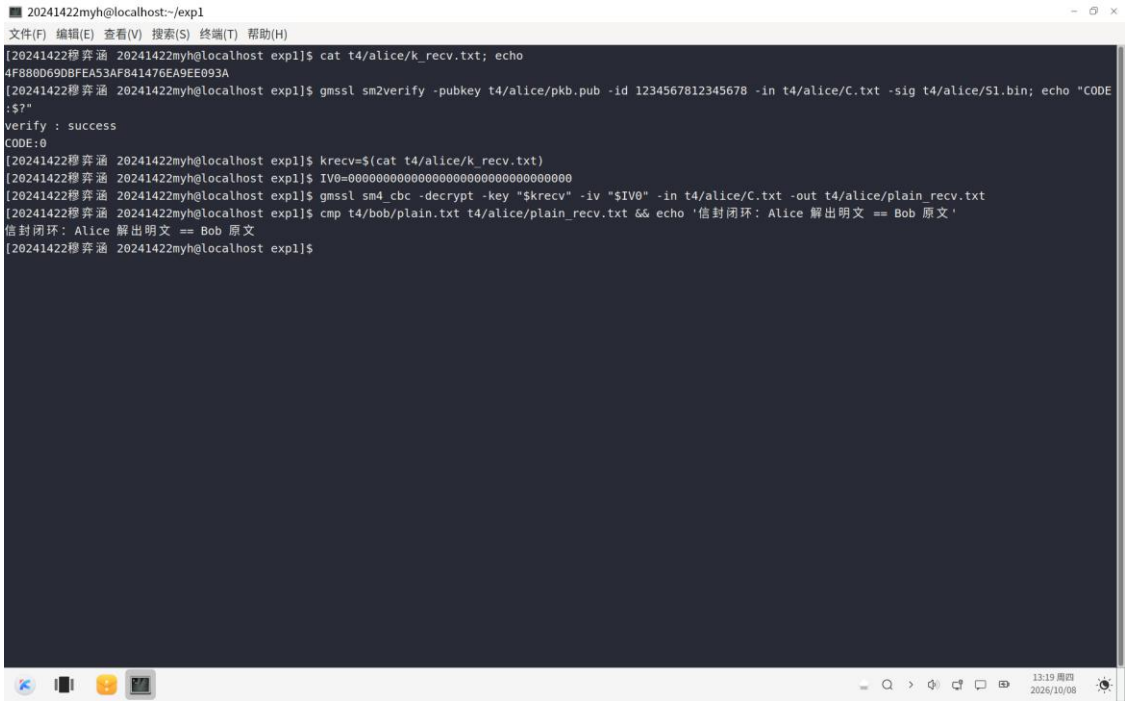


图 12 数字信封 GmSSL：Alice 侧验签与解信封闭环

2.6 步骤 6：托管到 GitHub 并记录 git log

2.6.1 操作内容

- `git remote add origin https://github.com/Yaqub-Saffar/lab1`，`git push -u origin master`；
- 每完成一项实践提交一次，`git log --oneline` 查看提交记录。

2.6.2 操作结果

课程项目仓库：`https://github.com/Yaqub-Saffar/lab1`。 `git log --oneline` 共 11 条提交（初始化 → 环境搭建 → 任务①OpenSSL → 任务②GmSSL → 任务③④数字信封 → README 回填链接，即图 13），按项留痕完整；其后报告定稿与格式转换另提交若干次，仓库现累计 16 条。

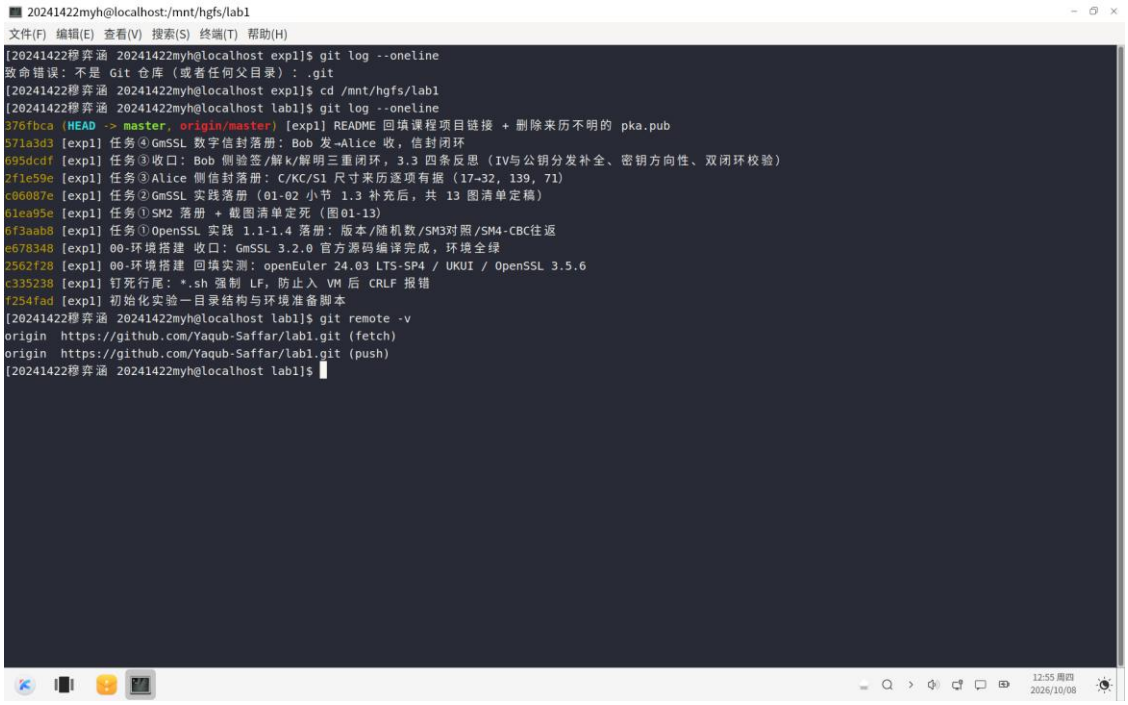


图 13 git log 与远端仓库

3. 实验体会

3.1 调试中出现的问题及解决过程

问题 1：gitee 镜像版 GmSSL 的 CMakeLists 与源码树自相矛盾，make install 中途失败。 现象：make 报 [100%] Built target gmssl 后，make install 报 file INSTALL cannot find "../GmSSL/thirdpart/gmssl/include/gmssl"。 排查：grep -rn thirdpart CMakeLists.txt cmake src 发现镜像的 CMakeLists.txt 引用了并不存在的 thirdpart/ 目录（也无 .gitmodules）；对照官方 master 的同一指令路径是正确的 include/gmssl。 解决：弃用镜像，改用官方 v3.2.0 源码包重新编译安装。**教训：make 成功 ≠ 安装成功，make install 报错要回源码找根因。**

问题 2：GmSSL 3.2.0 命令行与题面/旧文档语法不符。 现象：按题面写 `gmssl rand -hex 16` 报 `illegal option '16'`；按旧文档 `gmssl sm3 t.txt` 报 `illegal option 't.txt'`。 解决：读源码 `tools/rand.c`、`tools/sm3.c` 的 usage，改用 `gmssl rand -hex -outlen 16`、`gmssl sm3 -in t.txt`。**凡 GmSSL 命令一律先对 3.2.0 源码 usage 核对**，本实验 8 条命令核对后一次成功。

问题 3：SM2 签名的用户标识（distid）必须签/验两侧同值。 OpenSSL 侧不显式指定时用内置默认标识，GmSSL 侧需 `-id 1234567812345678`；两侧同值验签才成功，否则验签失败。数字信封中据此显式写定。

问题 4：同一份明文的 SM2 密文，OpenSSL 与 GmSSL 尺寸不同（139 vs 141）。 经 `od` 字节级比对，确认是 **DER 序列化结构差异**而非算法差异；因此本实验只做同工具闭环，不做跨库互解。

问题 5：篡改负例的"失败判定"不能依赖报错文案。 OpenSSL 报 `Signature Verification Failure`，GmSSL 报 `inner error` + 源码行号，文案完全不同，但语义一致（验签失败）。**判定依据统一取退出码 `exit=1`**，脚本校验应看返回值而非 stderr。

3.2 心得体会

通过本次实验，我掌握了在 openEuler 上用 OpenSSL 与 GmSSL 完成 SM3/SM4/SM2 实践以及带签名数字信封的完整流程，也真正体会到「先按源码 usage 核对命令」远比照抄文档可靠。GmSSL 与 OpenSSL 在命令行语法、SM2 用户标识以及 SM2 密文 DER 结构上的差异让我明白，密码工具的「同算法」并不等于「同接口」，跨库使用时必须逐字节核实。